

Preserving OCL Violation Identifiers over UML Property Graphs Companion Technical Report

Quoc-Bao Hoang Hoang-Viet La Trinh Duc-Hanh Dang

Version 0.4 – 20 September 2026

Abstract

This report gives the definitions, derivations, and evidence ledger omitted from the conference paper. It studies a restricted, typed profile of UML/OCL invariants over finite well-formed snapshots. A prescribed representation contract encodes a snapshot as a property graph. The metadata-parameterized expression translation T_Σ maps normalized Core expressions to the typed logical query language $\mathbb{Q}$, while $\text{Compile}_{S,\Sigma}$ names the presentation-level composition that maps an admitted invariant to generated read-only Cypher text, a result contract, and generated parameters. The central claim is observational: whenever observational representation adequacy and all admission/refinement premises are discharged, source evaluation and formal target evaluation return exactly the same stable identifiers of violating context objects. Concrete Neo4j runs are reported as a separate runtime-refinement obligation. The report therefore does not claim full OMG OCL coverage, arbitrary graph or Cypher support, performance, or formal verification of an implementation.

Contents

1	Introduction and Relationship to the Conference Paper	4
1.1	Purpose and Scope	4
1.2	Central Observational Claim	5
1.3	Notation and Semantic Boundaries	5
1.4	Evidence and Non-Claims	6
1.5	Relationship to the Conference Paper	7
2	UML Snapshot Model	7
2.1	Schema Domain	7
2.2	Finite Snapshots	8
2.3	Logical Navigation	8
2.4	Well-Formedness and Failure	8

3	Admitted OCL Profile	9
3.1	Surface Boundary	9
3.2	Typing and Undefinedness	9
3.3	Supported and Rejected Families	10
3.4	Normalization Boundary	11
4	Source and Core Semantics	11
4.1	Core Types and Expressions	11
4.2	Typing and Carriers	12
4.3	Denotational Semantics	12
4.4	Normalization Preservation	13
5	Property-Graph Representation and Construction	13
5.1	Representation Metadata	13
5.2	Construction Invariant	14
5.3	Graph Adequacy	14
5.4	Canonical Versus Adequate Encodings	15
6	Logical Query Language Q	16
6.1	Syntax and Typing	16
6.2	Graph Observers	16
6.3	Denotational Semantics	16
6.4	Violation Wrapper and Partial Results	17
6.5	Why Q Is Needed	17
7	Core-to-Q Translation and Admission	17
7.1	Translation Pipeline and Success	17
7.2	Constructor rules	18
7.3	Iterators, inheritance, and projection	19
7.4	Admission and primitive soundness	19
7.5	Translation diagnostics	19
8	Q-to-Cypher Realization and Serialization	20
8.1	Public Realization Interface	20
8.2	Formal Target and Witness	20
8.3	Internal Cypher AST and Serialization	21
8.4	Target Observation and Refinement Lemma	21
8.5	Concrete-Engine Boundary	22
9	Numeric Refinement and Certificates	22
9.1	Domains and carriers	22
9.2	Certificates	22
9.3	Acceptance matrix	23
9.4	Unsafe cases	23

10 End-to-End Correctness Proof	24
10.1 Correspondence Relations	24
10.2 Frontend and Core-to- Q Preservation	24
10.3 Violation Projection	25
10.4 Presentation-Level Composition	25
10.5 Main Theorem	25
10.6 Non-Circularity and Assurance Boundary	26
11 Mechanization and Proof Assurance	27
11.1 Build and trusted base	27
11.2 Coverage matrix	28
11.3 Gaps and stale artefacts	28
12 Prototype Boundary	28
12.1 Architecture	29
12.2 Traceability	29
12.3 Validation gates	29
12.4 Refinement gaps	29
13 Experimental Method	29
13.1 Research questions	30
13.2 Subjects and units	30
13.3 Oracles and coverage	30
13.4 Live protocol	31
14 Results	31
14.1 Headline outcomes	31
14.2 Risk-oriented outcomes	31
14.3 Threats to validity	32
15 Reproducibility and Traceability	32
15.1 Evidence snapshot and environment	32
15.2 Reproduction gates	33
15.3 Artifacts and regeneration	34
15.4 Interpretation of failures	34
A Core-to-Q Translation Rules	34
A.1 Explicit Surface Rejections	36
B Representative Proof-Obligation Ledger	36
C Generated Cypher Examples	38
C.1 ManagerSalaryDominates: Reverse Navigation and Three-Valued Fold	38
C.2 Bag Multiplicity	39

C.3 Rejected Numeric Case	40
C.4 Compact Compiler-Generated Attribute Read	40
D Experimental Inventory and Catalogue Schema	40

1 Introduction and Relationship to the Conference Paper

1.1 Purpose and Scope

The companion report makes the semantic contract behind the conference paper auditable. The paper presents the motivating counterexample, representative translation rules, the preservation lemmas, the end-to-end theorem, and the claim-oriented evaluation within the conference limit. This report expands those claims into definitions, an exhaustive Core-to-Q catalogue, representative Q-to-Cypher realization rules, a representative proof-obligation ledger, generated-query traces, and an aggregate experimental inventory with reproducibility records. It is not a second experimental paper and it does not enlarge the theorem merely because the report has space for more implementation details.

The admitted source terminology follows the OCL 2.4 specification [15]; USE provides a representative executable UML/OCL validation environment [10]. The target setting is informed by property-graph schema work [1] and the published account of Cypher [9]. Earlier OCL-to-relational-query translations provide useful comparison points [8, 13]; Nguyen and Clavel also study proof of correctness for SQL implementations of OCL constraints [14]. Radke et al. establish satisfaction equivalence between Essential OCL and nested graph constraints [16], while Graphiti studies query equivalence across relational and graph models relative to an explicit database transformer [11]. Neither result establishes the UML/OCL-to-Cypher violation-observation theorem studied here. Graph-backed model persistence and validation systems provide additional engineering context. Mogwai first translated OCL to Gremlin for scalable model queries and later moved validation and violation retrieval to the graph backend [6, 3]; the claim here is not query generation alone, but the shared representation contract and conditional preservation of the exact violation-ID observation [2, 5, 7]. Gremlin-ATL translates ATL transformations whose expressions use OCL to Gremlin [4], while UML-to-property-graph work provides a separate model-representation line [12]; neither changes the specific composition-correctness question addressed here.

The object of study is validation, not persistence by itself. A finite UML snapshot is a set of typed objects, slots, and association occurrences. An OCL invariant is evaluated over that snapshot and identifies context objects whose body is not true. The same snapshot is represented by a property

graph and the invariant is translated to a typed logical query and then to generated read-only Cypher. Correctness is judged by the public observation returned by the two evaluations: a set of application-level stable object identifiers.

1.2 Central Observational Claim

Let M be a finite well-formed UML snapshot, G its property-graph representation, and inv an invariant in the admitted profile. Let γ be the witness supplied by observational representation adequacy. The report uses the following conditional claim throughout:

If $\text{Adequate}_\Sigma(M, G)$ holds, and if frontend, normalization, primitive refinement, translation, realization, and serialization premises are discharged, then source OCL evaluation and the formal semantics of the generated read-only query return exactly the same set of stable identifiers for violating context objects.

For a context object o , violation means that the invariant body evaluates to anything other than the admitted truth value $\top$. Thus false and typed bottom are observable violations, while a defined empty collection is not confused with a bottom collection. The claim is intentionally observational: it does not assert that every internal graph node, property, or query plan is unique.

1.3 Notation and Semantic Boundaries

The notation is fixed before the detailed sections so that no implementation module or database API becomes an accidental semantic premise.

Concept	Notation	Meaning
UML snapshot	M	Finite, well-formed source state.
Graph instance	G	Concrete property graph used only for query interpretation.
Representation metadata	Σ	Keys, navigation, type/extent descriptors, and value/link encodings fixed by the prescribed representation.
Object correspondence	γ	Bijection between source objects and graph object observations.
Normalized source	Core	Typed source-expression language after frontend normalization.

Logical target	$\mathbf{Q}$	Typed query language with graph observers and three-valued carriers.
Expression translation	$T_{\Sigma}(e)$	Metadata-parameterized translation from Core to $\mathbf{Q}$; independent of G .
Compilation	$\text{Compile}_{S,\Sigma}(inv)$	Presentation-level composition of frontend, Core-to- $\mathbf{Q}$ translation, wrapper construction, and realization.
Source observation	$\text{Viol}_O(M, inv)$	Stable IDs of context objects whose body is not $\top$.
Target observation	$\text{Viol}_C(G, s; \pi, \Delta)$	Partial observation of generated text s under the complete parameters and result contract.

The distinction between Σ and G is essential. T_{Σ} chooses the keyed observers and typed query constructors from representation metadata; it does not inspect which objects or links happen to be present in G . Only $\llbracket q \rrbracket_{\mathbf{Q}}(G, \eta)$ and the resulting Viol_C consume a graph instance. This separation lets one translation be interpreted over multiple adequate instances of the same representation contract.

1.4 Evidence and Non-Claims

The report keeps five forms of evidence distinct:

1. mathematical statements and their hand derivations;
2. selected machine-checked proof kernels;
3. executable local differential comparisons;
4. explicit rejection cases at the admission boundary; and
5. live comparisons against one recorded Neo4j deployment.

The current evidence snapshot contains 199 local comparisons, 23 explicit rejections, and 85 live comparisons. Some positive rows exercise prototype-only extensions beyond the theorem profile; the aggregate counts therefore support the reported implementation behavior, not a larger theorem. They do not replace the theorem or establish performance. In particular, a successful live query is evidence for a runtime boundary, not a proof that all Neo4j releases implement the formal target semantics.

The theorem does not claim full OMG OCL equivalence, arbitrary graph encodings, arbitrary Cypher queries, unrestricted numeric behavior, universal Neo4j conformance, runtime or throughput guarantees, or formal verification of the complete implementation. Unsupported or uncertified inputs are rejected; they are never silently reinterpreted as bottom or as an empty violation set.

1.5 Relationship to the Conference Paper

The conference paper is the self-contained, page-limited statement of the result. This report supplies the material that was deliberately moved out of its main text:

- Sections 2–3 define the source snapshot domain and the admitted OCL boundary.
- Sections 4–9 give Core, graph, Q, realization, and numeric semantics.
- Section 10 expands the preservation lemma and theorem into a dependency proof and two inclusion arguments.
- Sections 11–15 separate mechanization, prototype boundary, experiment design, results, and reproducibility.
- Appendices A–D contain the exhaustive Core-to-Q rules, representative realization and proof-obligation ledgers, generated-query artifacts, and the aggregate experimental inventory/catalogue schema.

Any discrepancy between this report, the paper, and the evidence manifest is a release blocker. Counts, notation, and scope statements must be updated together rather than corrected locally in only one document.

2 UML Snapshot Model

2.1 Schema Domain

A UML schema is modelled by finite sets of classes, primitive types, attributes, associations, association ends, and generalization declarations. An association end r has a declaring association, a source class, a target class, a declared collection kind (to-one, Set, or Bag), and an association-end key. The key is semantic metadata: it remains stable even if a concrete graph implementation chooses another label or property spelling.

Generalization induces a directed relation $K_1 \prec K_2$ when K_1 is a direct subclass of K_2 . Its reflexive–transitive closure is written $\preceq$. The closure is used to define context extents rather than relying on a database label scan that happens to include inherited objects.

2.2 Finite Snapshots

Definition 2.1 (UML snapshot) *A snapshot M consists of the finite object set $Obj(M)$, a direct runtime type map $type_M : Obj(M) \rightarrow ClsKey_\Sigma$, an injective application-level identifier map $id_M : Obj(M) \rightarrow ID$, scalar slots, and association link occurrences. A link occurrence records its association-end key, endpoint objects, and admitted qualifier values.*

The direct type is not replaced by the transitive set of supertypes. For a class K , the inheritance-aware extent is

$$Inst_M(K) = \{o \in Obj(M) \mid type_M(o) \preceq K\}.$$

Each object occurs once in this set even when several inheritance paths reach the same declared type. An invariant with context K scans this extent and projects $id_M(o)$ rather than an internal position or database ID.

For each scalar declaration key and object, a well-formed snapshot has at most one slot occurrence. A slot is either defined with a value in its declared type or explicitly absent. A to-one navigation is well-defined only when its resolved occurrence count is exactly one; zero or multiple matches are represented by typed bottom at the semantic boundary. A Set-valued navigation removes duplicate support, while a Bag-valued navigation retains occurrence multiplicity.

2.3 Logical Navigation

The physical direction chosen to store an edge is not the meaning of an OCL association end. For each association-end key r , the snapshot defines a forward occurrence relation $fwd_M(r)$ and a reverse relation $rev_M(r)$ by endpoint roles. They are two views of the same keyed occurrences. A graph construction is adequate only if its observers recover both views, regardless of the physical arrow used by a backend.

2.4 Well-Formedness and Failure

The source domain excludes duplicate stable IDs, dangling link endpoints, ambiguous scalar slots, malformed declared types, and invalid qualifier payloads. It also requires finite object, slot, and link sets. Construction must return an explicit failure for a malformed snapshot. Failure is not coerced to an empty graph, because doing so would make a missing object indistinguishable from a valid empty extent and would invalidate the violation observation.

The graph adequacy relation used later is intentionally stronger than mere node-count equality. It requires a correspondence preserving stable IDs, direct types, inheritance-aware extents, typed slots, and keyed link occurrences in both logical directions. Section 5 proves that the prescribed

abstract construction specification is sufficient to establish this relation. The production graph builder remains outside the formalized transformation and is evaluated empirically, as in the conference paper.

3 Admitted OCL Profile

3.1 Surface Boundary

The report treats the admitted language as a typed profile mapped from a surface fragment of OMG OCL. It does not claim equivalence to all OMG OCL constructs. A source text contains one invariant declaration with a context class, a name, and a Boolean-valued body. Names are resolved against the UML schema and lexical iterator bindings before normalization.

The admitted expression grammar is summarized below:

$$\begin{aligned}
 e ::= & \ c \mid x \mid \text{let } x = e \text{ in } e \mid \text{if } e \text{ then } e \text{ else } e \\
 & \mid e.a \mid e.r \mid \text{uop}(e) \mid e \text{ bop } e \\
 & \mid e \rightarrow \text{forAll}(x \mid e) \mid e \rightarrow \text{exists}(x \mid e) \\
 & \mid e \rightarrow \text{select}(x \mid e) \mid e \rightarrow \text{collect}(x \mid e).
 \end{aligned}$$

Here c includes typed scalar and collection literals, a is a resolved attribute declaration, and r is a resolved association end. The admitted collection observations are **size**, **isEmpty**, **notEmpty**, and **sum**. Boolean, typed equality/comparison, and certified numeric operations are also included. Appendix A gives one catalogue row for every constructor family in this profile.

The body of **collect** must have a non-collection element type; the result is a flat Bag and preserves source occurrences. The indeterminate choice operation **any** is not in the grammar. In particular, this profile neither chooses a database-dependent witness nor assumes that the satisfying element is unique; surface uses of **any** are rejected.

The spellings **asSet** and **asBag**, type tests and casts, **allInstances**, **reject**, membership/count operations, and Set union/intersection are not part of the theorem profile. Mentioning Set and Bag carriers does not implicitly admit these operations.

3.2 Typing and Undefinedness

Typing judgments have the form $\Gamma \vdash e : \tau$, where Γ records the type of ‘self’, iterator variables, and parameters. A resolved association end determines both its target class and its collection kind; a written role name is not sufficient to type a navigation.

The profile uses typed bottom to represent an undefined result at a particular type. Typed bottom is distinct from a defined empty Set or Bag. At Boolean type, evaluation uses the carrier $D_B^\perp = \{\top, \text{F}, \perp_B\}$. The iterator

fold satisfies

$$\bigwedge_3 [] = \mathbf{T}, \quad \mathbf{F} \wedge_3 x = \mathbf{F}, \quad \mathbf{T} \wedge_3 x = x,$$

with $\perp_B \wedge_3 \mathbf{T} = \perp_B$. Thus ‘forAll’ over a defined empty collection is true, whereas a bottom collection is bottom. This distinction is part of the admitted semantics and is carried into Q.

The admitted bottom is a semantic carrier for the generated profile; this report does not silently identify it with every occurrence of OMG OCL ‘null’ or ‘invalid’. Operations whose observable behavior depends on distinctions not represented by the carrier are outside the profile and are rejected at admission. A full comparison with OMG OCL’s undefinedness taxonomy belongs in the supplementary profile matrix, not in an unqualified theorem premise.

3.3 Supported and Rejected Families

Family	Admitted cases	Boundary/rejection condition
Literals and variables	Typed scalar, object, Set, and Bag literals; lexical variables	Ill-typed or unbound terms reject.
Attributes	Declaration-keyed scalar reads; an absent snapshot slot denotes typed bottom	Unresolved or duplicate declaration, non-unique/malformed physical slots, or uncertified codec fails admission/adequacy.
Navigation	To-one, Set, and Bag; forward and reverse logical roles	An unresolved end or wrong receiver rejects.
Iterators	Flat finite ‘forAll’, ‘exists’, ‘select’, and ‘collect’ with non-collection body, including nested lexical scopes	‘any’, ‘reject’, unsupported iterators, collection-valued ‘collect’ bodies, or capture ambiguity reject.

Collections	Empty collections, Set support, Bag occurrences, ‘size’, ‘isEmpty’, ‘notEmpty’, and ‘sum’	Ordered/nested collections, membership/count operations, conversion, and Set union/intersection reject.
Inheritance	Context extents and inheritance-aware navigation	Explicit type tests, casts, and ‘allInstances’ are outside the theorem profile.
Numeric operations	Operations with discharged exact-refinement premise	Unsafe range, overflow, coercion, or rounding case rejects.

The admission predicate is static: it checks profile membership, successful resolution and typing, and the primitive/refinement certificates required by the target constructors. It is not defined by comparing source and target results and therefore does not make the preservation theorem circular.

3.4 Normalization Boundary

Frontend normalization resolves names, makes receiver and end declarations explicit, introduces capture-avoiding iterator bindings, and lowers admitted surface terms to **Core**. The normalization lemma in Section 4 states that this lowering preserves the source denotation. A failed parse, resolution, typing, or admission check returns a diagnostic; it does not produce a partial query that could be mistaken for a valid empty result.

4 Source and Core Semantics

4.1 Core Types and Expressions

The Core language is the normalized source language used by the preservation proof. Normalization removes surface-name ambiguity but does not evaluate an expression. A Core term therefore retains the resolved declaration key, receiver type, collection kind, and lexical binding needed by translation.

The type grammar is

$$\tau ::= \text{Bool} \mid \text{String} \mid \text{Integer} \mid \text{Real} \mid \text{Obj}(K) \mid \text{Set}(\tau) \mid \text{Bag}(\tau).$$

Each type τ has a distinct typed-bottom value $\perp_\tau$; typed bottom is a value, not an additional type constructor. In particular, $\perp_{\text{Set}(\tau)}$ must not be confused with a defined $\text{Set}(\tau)$ containing zero elements.

The paper-level Core calculus separates value expressions from occurrence-oriented collection plans:

$$\begin{aligned} e_C ::= & c_\tau \mid \perp_\tau \mid x \mid \text{Let}(x, e_C, e_C) \mid \text{If}(e_C, e_C, e_C) \\ & \mid \text{Attr}(e_C, a) \mid \text{NavOne}(e_C, r) \mid \text{Un}_{o_1}(e_C) \mid \text{Bin}_{o_2}(e_C, e_C) \\ & \mid \text{Lit}_\kappa(\bar{e}_C) \mid \text{Materialize}(P_C) \mid \text{ForAll}(P_C, x, e_C) \mid \text{Exists}(P_C, x, e_C), \\ P_C ::= & \text{From}(e_C) \mid \text{Scan}_K \mid \text{NavMany}_\kappa(e_C, r) \mid \text{Select}(P_C, x, e_C) \\ & \mid \text{Collect}(P_C, x, e_C) \mid \text{Distinct}(P_C) \mid \text{LetP}(x, e_C, P_C). \end{aligned}$$

Here a, r, K are resolved keys, $\kappa \in \{\text{Set}, \text{Bag}\}$, o_1 ranges over the admitted unary and collection-observation operators, and o_2 ranges over Boolean, typed equality/comparison, and certified numeric binary operators. This is the same calculus used in the main paper; unsupported constructors never enter Core.

4.2 Typing and Carriers

Typing has the form $\Gamma \vdash e : \tau$. The context Γ maps ‘self’, iterator variables, and parameters to types. Representative rules are:

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \quad \frac{\Gamma \vdash e : \text{Obj}(K) \quad a : K \rightarrow \tau}{\Gamma \vdash \text{Attr}(e, a) : \tau}$$

and

$$\frac{\Gamma \vdash e : \text{Set}(\text{Obj}(K)) \quad \Gamma, x : \text{Obj}(K) \vdash b : D_B^\perp}{\Gamma \vdash \text{ForAll}(e, x, b) : D_B^\perp}.$$

The corresponding Bag rule preserves Bag multiplicity in the input carrier. Type checking rejects a navigation whose receiver does not resolve to the declaring class of its association end.

For each type τ , let V_τ be its defined carrier and add a distinct bottom element $\perp_\tau$. Boolean values use $D_B^\perp = \{\top, \text{F}, \perp_B\}$. Object values are source objects; their target counterparts are related by γ . Set values are finite supports, while Bag values are finite occurrence lists. A value is therefore either a defined member of V_τ or $\perp_\tau$.

4.3 Denotational Semantics

Evaluation is written $\llbracket e \rrbracket_{\text{Core}}(M, \rho)$, where ρ is a type-correct environment. Scalar literals, variables, and homomorphic operators follow their ordinary typed interpretation. The non-homomorphic clauses are the semantic core. The source notation $\llbracket \varphi \rrbracket_O$ is reserved for the surface OCL body before front-end normalization.

For an attribute key a , the attribute clause reads the unique slot of the receiver. A missing or ambiguous slot yields $\perp_{\tau(a)}$; a defined slot returns its typed payload. For navigation, the receiver and qualifiers are evaluated first. A bottom receiver yields bottom. A to-one result is defined only for exactly one matching occurrence; zero or multiple occurrences yield typed bottom. A Set result is the support of matching endpoints and a Bag result is the endpoint occurrence list.

The three-valued conjunction used by iterators is:

$\wedge_3$	T	F	$\perp_B$
T	T	F	$\perp_B$
F	F	F	F
$\perp_B$	$\perp_B$	F	$\perp_B$

For a defined finite collection $[v_1, \dots, v_n]$ and body b ,

$$\llbracket \text{ForAll}(P, x, b) \rrbracket_{Core} = \bigwedge_{j=1}^n \llbracket b \rrbracket_{Core}[\rho[x \mapsto v_j]],$$

with the empty fold equal to T. If P itself is bottom, the result is bottom before the body is evaluated. ‘Exists’ uses the dual $\vee_3$ fold. Bag occurrences are visited separately, even when their endpoint values are equal.

An invariant $inv = (K, \varphi)$ has source observation

$$\text{Viol}_O(M, inv) = \{id_M(o) \mid o \in \text{Inst}_M(K), \llbracket \varphi \rrbracket_O(M, \rho_o) \neq T\}.$$

The set projection removes duplicate paths but never removes a distinct stable identifier.

4.4 Normalization Preservation

The public frontend operation is Front_S . Its preservation obligation is

$$\text{Front}_S(inv) = \text{Success}(K, e) \implies \llbracket \varphi \rrbracket_O(M, \rho) = \llbracket e \rrbracket_{Core}(M, \rho).$$

The proof is structural induction over the resolved surface syntax. The navigation case uses the resolved association-end key; the iterator case extends the environment with a fresh variable and invokes alpha-renaming to avoid capture. Collection lowering preserves support for Sets and occurrence lists for Bags. This lemma is what allows Section 10 to start its main induction from Core rather than from parser syntax.

5 Property-Graph Representation and Construction

5.1 Representation Metadata

The prescribed representation is described at the logical level by metadata Σ . It contains declaration keys for attributes and association ends, stable-ID

and direct-type fields, extent descriptors, and encoding/decoding functions for scalar values, definedness, qualifiers, and link occurrences. The metadata is fixed by the schema contract and is independent of a concrete instance G .

A graph instance is a finite property graph with object observations, keyed scalar slots, and keyed association occurrences. The physical labels, relationship direction, and internal node identifiers are representation choices; they are not themselves the semantic observation. Logical observers recover forward and reverse association-end views from the occurrence key and its endpoint roles.

5.2 Construction Invariant

The abstract construction Build_Σ takes a well-formed snapshot M and returns either an explicit failure or $\text{Success}(G)$. Its invariant has four cases:

1. Each source object creates exactly one graph object observation carrying the same stable ID and direct runtime type.
2. Each scalar slot creates one declaration-keyed observation. Definedness and value decoding round-trip at the declared type.
3. Each association occurrence creates one keyed graph occurrence with the same endpoint and qualifier data. Forward and reverse observers select the same occurrence from opposite roles.
4. The schema closure records every inherited extent membership once, after duplicate paths are removed.

Malformed input, duplicate stable IDs, dangling endpoints, or ambiguous scalar slots cause construction failure. In particular, failure is not encoded as an empty graph, because that would make a construction error observationally indistinguishable from a valid empty extent.

5.3 Graph Adequacy

Let $\text{Obj}(M)$ be the source-object set and let $\text{ObjObs}_\Sigma(G)$ be the image of the logical object observer. The latter excludes schema, metadata, and reified-association nodes even when they occur in the physical property graph.

Definition 5.1 (Graph adequacy) *For snapshot M and graph G , $\text{Adequate}_\Sigma(M, G)$ holds iff there is a bijection $\gamma : \text{Obj}(M) \cong \text{ObjObs}_\Sigma(G)$ such that, for every admitted object o , class key K , scalar declaration a , and association end r ,*

$$\begin{aligned} id_G(\gamma(o)) &= id_M(o), & type_G(\gamma(o)) &= type_M(o), \\ \hat{\gamma}(\text{slot}_M(o, a)) &= \text{slot}_G(\gamma(o), a), \\ \hat{\gamma}(\text{Nav}_M(o, r)) &= \text{Nav}_G^\Sigma(\gamma(o), r), & \gamma[\text{Inst}_M(K)] &= \text{Extent}_G^\Sigma(K). \end{aligned}$$

The slot correspondence includes definedness and the declared scalar codec; the navigation equality includes qualifiers and both logical directions. Equivalently, these equations preserve:

1. *stable identifiers and direct runtime types;*
2. *inheritance-aware context extents;*
3. *typed scalar slot observations and definedness; and*
4. *keyed association occurrences, qualifiers, and both logical directions.*

Set observers preserve support and Bag observers preserve occurrence counts. Qualifier payloads are structural snapshot data; the theorem profile does not add source-level qualified-navigation or association-class operators.

The lifting $\hat{\gamma}$ fixes primitive values and typed bottom, maps objects by γ , preserves Set support, and maps every Bag occurrence.

Proposition 5.2 (Construction establishes adequacy)

$$\text{Build}_{\Sigma}(M) = \text{Success}(G) \implies \text{Adequate}_{\Sigma}(M, G).$$

Proof. Define $\gamma(o)$ to be the unique graph object observation created for o . Case 1 gives bijectivity, identity, and direct type. Case 2 gives the typed slot round trip. Case 3 gives a one-to-one occurrence map and therefore equality of forward and reverse navigation. Case 4 gives the inheritance-aware extents after duplicate removal. These cases establish all adequacy conjuncts. No premise refers to the result of evaluating an OCL expression, so the construction obligation is independent of the preservation theorem.

5.4 Canonical Versus Adequate Encodings

The prototype uses one canonical encoding, but the theorem quantifies over any graph instance satisfying the logical adequacy contract. Two encodings may use different labels or physical edge directions and still be adequate if their observers expose the same keyed observations. Conversely, a graph with the right object count is not adequate if it loses declaration identity, reverse navigation, multiplicity, or stable IDs. This distinction prevents implementation-specific node layout from becoming an unstated theorem claim. This proposition concerns the abstract construction specification. The production graph builder is not formally verified; the main theorem therefore uses $\text{Adequate}_{\Sigma}(M, G)$ directly as premise P2.

6 Logical Query Language Q

6.1 Syntax and Typing

Q is a typed logical query language between Core and concrete Cypher. It has no database-specific node identifiers and no implicit evaluation strategy. Its value expressions e_Q and occurrence-oriented plans P_Q are:

$$\begin{aligned} e_Q ::= & c_\tau \mid \perp_\tau \mid x \mid \text{Let}(x, e_Q, e_Q) \mid \text{If}(e_Q, e_Q, e_Q) \\ & \mid \text{Attr}(e_Q, a) \mid \text{NavOne}(e_Q, r) \mid \text{Un}_{o_1}(e_Q) \mid \text{Bin}_{o_2}(e_Q, e_Q) \\ & \mid \text{Lit}_\kappa(\bar{e}_Q) \mid \text{Materialize}(P_Q) \mid \text{ForAll3}(P_Q, x, e_Q) \mid \text{Exists3}(P_Q, x, e_Q), \\ P_Q ::= & \text{FromCollection}(e_Q) \mid \text{ScanClass}_K \mid \text{NavMany}_\kappa(e_Q, r) \\ & \mid \text{Filter}(P_Q, x, e_Q) \mid \text{Collect}(P_Q, x, e_Q) \mid \text{Distinct}(P_Q) \mid \text{PlanLet}(x, e_Q, P_Q). \end{aligned}$$

The query wrapper additionally contains a context key, a result projection, and a stable-ID result contract. Typing judgments $\Gamma_Q \vdash q : \tau$ mirror Core types, but graph observers make the representation metadata explicit. For example, ‘NavMany $_\kappa$ ’ preserves Set support or Bag occurrences according to κ , while ‘NavOne’ returns either one typed object or typed bottom. The constructor vocabulary used throughout this report is **Attr**, **NavOne**, and **NavMany**.

6.2 Graph Observers

An observer reads the logical graph interface induced by Σ :

$$\begin{aligned} \text{Extent}_G^\Sigma(K) &= \text{objects in the inheritance-aware extent of } K, \\ \text{slot}_G(o, a) &= \text{the keyed typed slot observation for } a, \\ \text{links}_G(o, r, \bar{u}) &= \text{matching keyed occurrences for end } r. \end{aligned}$$

The observer relation is defined by endpoint roles rather than physical edge orientation. A missing to-many match returns a defined empty collection; a to-one observer checks exact cardinality and returns typed bottom for zero or more than one match. The observer interface is the only place where G enters the logical denotation.

6.3 Denotational Semantics

Evaluation is written $\llbracket q \rrbracket_Q(G, \eta)$. Attribute reads decode the keyed slot and preserve definedness. Navigation first evaluates the receiver and qualifiers, then applies the observer relation. Scalar and Boolean operators are homomorphic over corresponding typed values, using the same $D_B^\perp$ tables as Core.

For a defined finite query collection $P = [v_1, \dots, v_n]$,

$$\llbracket \text{ForAll3}(P, x, b) \rrbracket_Q = \bigwedge_{3, j=1}^n \llbracket b \rrbracket_Q[\eta[x \mapsto v_j]], \quad \bigwedge_3[] = \top.$$

If $P = \perp_{\text{Set}(\tau)}$ or $\perp_{\text{Bag}(\tau)}$, the result is $\perp_B$. Bag occurrences remain distinct during the fold. ‘Exists3’ uses the corresponding disjunction fold. These clauses are deliberately identical to Core clauses; the expression-preservation lemma proves that the input values to the folds correspond pointwise.

6.4 Violation Wrapper and Partial Results

For context class K and body query q , the wrapper $\text{Violations}(K, q)$ is a closed Q query: it binds the context variable before evaluating q . Its denotation is

$$\llbracket \text{Violations}(K, q) \rrbracket_Q(G) = \{id_G(o) \mid o \in \text{Extent}_G^\Sigma(K), \llbracket q \rrbracket_Q(G, \eta_o) \neq \top\}.$$

Here η_o binds **self** to the graph observation of o ; $\text{Extent}_G^\Sigma(K)$ includes direct instances and subclass instances, and $id_G(o)$ is the stable application identifier. The result contract requires a Set of application-level stable IDs and specifies the public parameter domain. It does not expose graph-internal node IDs. A malformed row, missing stable ID, parameter error, parse error, or execution error is a failure result, not an empty Set. This partiality is important for the theorem: only successful target evaluation contributes a target observation.

6.5 Why Q Is Needed

Direct induction from Core to arbitrary Cypher would mix source semantics with syntax, backend coercions, and query-engine behavior. Q isolates the semantic kernel: typed observers, typed-bottom propagation, collection folds, and violation projection. Section 7 proves Core-to-Q preservation; Section 8 handles realization and serialization separately. The decomposition makes the theorem auditable without claiming that all of Cypher is modeled.

7 Core-to-Q Translation and Admission

This section defines the source-to-logical-query translation. The translation is parameterised by representation metadata Σ ; it is therefore fixed before a graph instance is selected. A graph G occurs only when the translated Q term or generated Cypher is evaluated.

7.1 Translation Pipeline and Success

For a source invariant inv , the frontend resolves names and types, makes iterator binders and declarations explicit, and returns its context key K and

normalized Core body e . Translation consumes only e and the representation specification Σ :

$$\text{Front}_S(\text{inv}) = \text{Success}(K, e), \quad T_\Sigma(e) = \text{Success}(q).$$

The wrapper and realization stages are

$$\text{Realize}_\Sigma(\text{Violations}(K, q)) = \text{Success}(s, \Delta, \pi_{\text{gen}}).$$

Any stage can instead return a diagnostic failure. Failure is observable: no generated Cypher and no empty violation set are returned. Admission is thus a static predicate on e and Σ , not a test comparing source and target results.

Definition 7.1 (Translation judgment) The judgment is written

$$\Sigma \vdash e \rightsquigarrow q.$$

It is the successful translation relation, defined by $\Sigma \vdash t \rightsquigarrow t'$ iff $T_\Sigma(t) = \text{Success}(t')$, for both expression and plan sorts. The compiler checks syntactic, resolution, typing, and static admission conditions. Any remaining semantic side condition is an explicit premise of preservation; it is not silently discharged by compiler success.

7.2 Constructor rules

The following rules expose the representative recursive shape of the translation; Appendix A gives the exhaustive constructor and operation catalogue. In a rule's right-hand side, $T_\Sigma(e_j)$ denotes the query payload of a successful recursive translation; success of every recursive call is a side condition. A failed call has no translation judgment and cannot be embedded as a query node.

$$\begin{array}{ll} \Sigma \vdash x & \rightsquigarrow x \\ \Sigma \vdash c_\tau & \rightsquigarrow c_\tau \\ \Sigma \vdash \text{Bin}_{o_2}(e_1, e_2) & \rightsquigarrow \text{Bin}_{o_2}(T_\Sigma(e_1), T_\Sigma(e_2)) \\ \Sigma \vdash \text{Attr}(e, a) & \rightsquigarrow \text{Attr}(T_\Sigma(e), a) \\ \Sigma \vdash \text{NavOne}(e, r) & \rightsquigarrow \text{NavOne}(T_\Sigma(e), r) \\ \Sigma \vdash \text{NavMany}_\kappa(e, r) & \rightsquigarrow \text{NavMany}_\kappa(T_\Sigma(e), r) \\ \Sigma \vdash \text{ForAll}(P, x, b) & \rightsquigarrow \text{ForAll3}(T_\Sigma(P), x, T_\Sigma(b)) \\ \Sigma \vdash \text{Exists}(P, x, b) & \rightsquigarrow \text{Exists3}(T_\Sigma(P), x, T_\Sigma(b)). \end{array}$$

The receiver type determines navigation direction. Reverse navigation is translated by the reverse observer recorded in Σ , rather than by guessing the direction of a stored edge. The translation preserves Set versus Bag constructors and inserts an explicit deduplication node for violation projection.

7.3 Iterators, inheritance, and projection

Iterator binders are alpha-renamed before translation. A nested iterator gets a fresh logical variable, so a binder in an inner body cannot capture the receiver of an outer iterator. The context extent observer is inheritance-aware: an object contributes exactly once when its direct type is the context class or a permitted subtype.

The invariant wrapper evaluates the translated body for every object in the context extent and projects the stable identifier of every result different from $\top$. Projection is Set-valued even when an intermediate iterator is Bag-valued. The closed query $\text{Violations}(K, q)$ has the denotation defined in Section 6: each context object whose body result is not $\top$ contributes its stable ID exactly once, even if several link occurrences led to that result.

7.4 Admission and primitive soundness

Each primitive p has a checker Check_p with three outcomes: accept, reject, or accept under a certificate. The checker is structural. It verifies operand types, collection kind, definedness premises, and (for numeric operations) a certificate described in Section 9.

Lemma 7.2 (Primitive soundness) *If $\text{Check}_p(\tau_1, \dots, \tau_n) = \text{accept}$ and the value correspondence premises hold for the operands, then the source operation and its translated Q primitive produce corresponding values, including the same typed-bottom cases.*

The proof is by cases on the primitive table. Boolean cases use the three-valued table of Section 4; collection cases preserve support and multiplicity; navigation cases use graph adequacy; numeric cases invoke the certificate theorem. Because a rejected primitive has no translation derivation, the checker is fail-closed and cannot silently broaden the theorem’s language.

7.5 Translation diagnostics

Diagnostics identify the first failed obligation. Categories include name or type errors, unsupported iterators, missing direction metadata, illegal casts, collection mismatches, and missing numeric certificates. This taxonomy is part of the result contract and is useful in the rejection catalogue, but it does not alter the semantic theorem. In all successful derivations, T_Σ depends on Σ only; G is introduced at $\llbracket q \rrbracket_Q(G, \eta)$.

8 Q-to-Cypher Realization and Serialization

8.1 Public Realization Interface

The public boundary is exactly the one used in the conference paper:

$$\begin{aligned} \text{Realize}_\Sigma(\text{Violations}(K, q)) &= \text{Success}(s, \Delta, \pi_{\text{gen}}), \\ \Delta &= (\text{Dom}_{\text{pub}}, \text{idCol}, \text{decode}). \end{aligned}$$

Here s is generated read-only Cypher text, π_{gen} contains generated bindings, and Δ specifies the public-parameter domain, returned stable-ID column, and decoder. The domains of π_{public} and π_{gen} must be disjoint; a collision is a realization failure. Realization reads q and Σ , not the graph instance G .

The emitted fragment uses MATCH patterns with WHERE predicates, WITH, UNWIND, RETURN, CASE, comprehensions, quantified predicates, and EXISTS/COLLECT subqueries. Writes, procedures, dynamic labels, and OPTIONAL MATCH are outside the profile. Failure at realization, parsing, execution, parameter validation, or decoding is never interpreted as an empty violation-ID set.

8.2 Formal Target and Witness

Let Target_Σ be the set of typed, read-only target-realization records. A record $p \in \text{Target}_\Sigma$ contains a target Boolean body, an ID-projection predicate, and serialization metadata. It is the abstract formal target of the main paper and is not itself a Cypher AST. The relation

$$\text{Witness}_\Sigma(p; G, K, q, s, \Delta, \pi_{\text{gen}})$$

holds exactly when p is the well-typed realization of $\text{Violations}(K, q)$ under Σ and the internal AST/serialization refinement described below yields precisely $(s, \Delta, \pi_{\text{gen}})$. The graph G fixes the semantic interpretation of p ; it is not read by Realize_Σ .

Define

$$E := \text{Extent}_G^\Sigma(K), \quad B_Q(g) := \llbracket q \rrbracket_Q(G, \eta_g),$$

where $\eta_g(\text{self}) = g$. The witness determines

$$E \subseteq \text{ObjObs}_\Sigma(G), \quad B_T^p : E \rightarrow D_B^\perp, \quad \text{Ret}^p : \text{ID} \rightarrow \text{Prop}.$$

The three refinement obligations are

$$\begin{aligned} \mathbf{R1} : & \quad \forall g \in E, \quad B_Q(g) = B_T^p(g), \\ \mathbf{R2} : & \quad \forall j \in \text{ID}, \quad \text{Ret}^p(j) \iff \exists g \in E : \text{id}_G(g) = j \wedge B_T^p(g) \neq \top, \\ \mathbf{R3} : & \quad \forall j \in \text{ID}, \quad j \in I \iff \text{Ret}^p(j), \end{aligned}$$

where R3 is stated only after successful execution and decoding have produced $I \subseteq \text{ID}$. R1 relates Q evaluation to the formal target; R2 connects target evaluation to exact stable-ID projection; R3 is the runtime/decoder refinement obligation.

8.3 Internal Cypher AST and Serialization

The report refines a target record through an internal proof-level Cypher AST:

$$p \in \text{Target}_\Sigma \xrightarrow{\text{LowerAST}_\Sigma} a \in \text{CypherAS} \xrightarrow{\text{Serialize}} s \in \text{CypherText}.$$

These are distinct internal objects, not alternative names for Realize_Σ , p , or s . The production CypherAS profile contains MATCH, WITH, UNWIND, CASE, expression comprehensions, EXISTS/COLLECT subqueries, RETURN, and the typed expressions required by the public fragment.

Representative lowering schemes preserve the constructor decisions already made by Q : attributes use checked property observations; to-one navigation uses a bounded match and cardinality guard; to-many navigation uses Σ -directed matching and preserves Set support or Bag occurrences; ForAll3/Exists3 use explicit counts and three-valued tags; and $\text{Violations}(K, q)$ scans the inheritance-aware extent, retains results different from T , and projects DISTINCT stable IDs.

Let

$$\text{parse}_5 : \text{CypherText} \longrightarrow \text{Result}(\text{CypherAS})$$

be the proof-level parser for the canonical read-only Cypher 5 grammar. The serialization obligation is

$$\text{Serialize}(a) = \text{Success}(s), \quad \text{parse}_5(s) = \text{Success}(a),$$

read modulo canonical alpha-normalization when fresh binders are introduced. Formal text semantics is the partial composition

$$\llbracket s \rrbracket_C(G; \pi, \Delta) = \begin{cases} \text{decode}_\Delta(\text{Eval}_{\text{gCy}}(G, a, \pi)), & \text{parse}_5(s) = \text{Success}(a), \\ \text{Failure}, & \text{parse}_5(s) = \text{Failure}. \end{cases}$$

Thus the main paper's text-level semantics is defined through the internal AST; arbitrary strings receive no generated-query semantics.

The production serializer has weaker evidence than the formal round trip. Neo4j's Cypher 5 parser accepts the emitted text, and structural checks cover the dialect header, single-query/RETURN shape, bounded relationships, and absence of forbidden constructs. These checks do not reconstruct the formal CypherAS node and therefore do not establish the production round trip.

8.4 Target Observation and Refinement Lemma

For a complete parameter map π and contract Δ , define

$$\text{Viol}_C(G, s; \pi, \Delta) = I \iff \llbracket s \rrbracket_C(G; \pi, \Delta) = \text{Success}(I).$$

This partial function is undefined on parsing, parameter, execution, or decoding failure.

Lemma 8.1 (Generated-query refinement) *Assume*

$$\text{Realize}_\Sigma(\text{Violations}(K, q)) = \text{Success}(s, \Delta, \pi_{\text{gen}})$$

and there is a p such that $\text{Witness}_\Sigma(p; G, K, q, s, \Delta, \pi_{\text{gen}})$ and R1–R2 hold. For $\pi_{\text{public}} \in \text{Dom}_{\text{pub}}$ with disjoint parameter names, let $\pi = \pi_{\text{public}} \uplus \pi_{\text{gen}}$. If $\llbracket s \rrbracket_C(G; \pi, \Delta) = \text{Success}(I)$ and R3 holds for p, I , then

$$I = \text{Obs}_Q(G, K, q).$$

Proof. R1 preserves the Boolean body for every object in E . R2 selects exactly the stable IDs whose target result is not $\top$, and R3 gives the same membership condition for the decoded set I . Extensionality therefore gives the claimed equality.

8.5 Concrete-Engine Boundary

A live Neo4j run supplies evidence for one recorded version, edition, and configuration. It checks that generated text parses, executes, and decodes to the expected result on the supplied fixture. Such a run supports R3 for the finite corpus; it cannot establish universal Neo4j conformance or discharge R3 for every input.

9 Numeric Refinement and Certificates

Numeric operations are the main place where mathematical values, host language values, and database values can diverge. The admitted profile keeps this risk explicit through range certificates rather than treating a finite test suite as a universal proof.

9.1 Domains and carriers

The source domain contains mathematical integers $\mathbb{Z}$ and finite real values. For the admitted concrete target, every Integer input and intermediate must be certified representable as a signed 64-bit integer. A Real operation is admitted only with a finite-representation and rounding certificate for the selected target carrier; NaN, infinity, implicit mixed-carrier coercions, and implementation-specific sentinels are rejected. Thus the formal theorem uses exact operations on the admitted certified domain. Agreement of the Java and Neo4j implementations with those carriers is a separate refinement boundary, not a consequence of the range-certificate theorem.

9.2 Certificates

A certificate is a finite record containing an interval for each numeric subexpression and the operation-specific side conditions. Intervals are propagated

compositionally; for example, addition uses endpoint sums and division additionally requires that zero is absent from the divisor interval. The checker returns `accept` only when every intermediate interval fits the target carrier and the operation has the same definedness behavior.

Definition 9.1 (Certificate soundness) Let c certify expression e . The certificate is sound when every source environment satisfying the recorded input intervals yields a defined source value whose target conversion is defined and equal to the target operation’s value.

Theorem 9.2 (Range-certificate soundness) If

$$\text{CertCheck}(e, c) = \text{accept},$$

then c is sound. Moreover, the numeric fragment of e satisfies the value-correspondence premise of primitive soundness.

The proof is induction on the expression tree. Base cases check literal bounds and declared ranges. Inductive cases use interval arithmetic and the side conditions for division, modulo, coercion, equality, and aggregation. The theorem is conditional on the certificate; it does not infer ranges from observed test data.

9.3 Acceptance matrix

Operation	Status	Required premise
Integer literal, addition, subtraction	<code>accept</code>	interval within target bounds
Integer multiplication	<code>conditional</code>	product interval within bounds
Division or modulo	<code>conditional</code>	non-zero divisor and exact definedness
Real literal and comparison	<code>conditional</code>	finite representation and rounding certificate
Mixed integer/real coercion	<code>conditional</code>	explicit conversion policy
Unbounded or unknown range	<code>reject</code>	no sound certificate available

9.4 Unsafe cases

Compiling an unbounded multiplication can overflow the target integer while the source remains defined. Likewise, a real comparison near a rounding boundary can change truth value after conversion. These are rejection examples, not failures of the main theorem: no successful translation is produced without the missing premise. The executable checker and the Lean

numeric families provide two witnesses for the same certificate idea; neither is presented as a proof of all Java or Neo4j arithmetic.

10 End-to-End Correctness Proof

This section expands the boundary-by-boundary argument from the conference paper without changing its theorem, notation, or admitted profile.

10.1 Correspondence Relations

Whenever $\text{Adequate}_\Sigma(M, G)$ is assumed, fix one witness $\gamma : \text{Obj}(M) \cong \text{ObjObs}_\Sigma(G)$ and its lifting $\hat{\gamma}$. Write

$$v \simeq_\Sigma w \quad :\Longleftrightarrow \quad w = \hat{\gamma}(v),$$

and

$$\rho \simeq_\Sigma \eta \quad :\Longleftrightarrow \quad \text{dom}(\rho) = \text{dom}(\eta) \wedge \forall x \in \text{dom}(\rho), \rho(x) \simeq_\Sigma \eta(x).$$

Thus primitives and typed bottom are fixed, objects correspond through γ , Sets correspond by support, and Bags correspond occurrence by occurrence. A typed-bottom collection never corresponds to a defined empty collection.

10.2 Frontend and Core-to-Q Preservation

Lemma 10.1 (Frontend preservation) *Assume $\text{Conforms}(M, S) \wedge \text{WF}_S(M)$ and $\text{Front}_S(\text{inv}) = \text{Success}(K, e)$. For every $o \in \text{Inst}_M(K)$,*

$$\llbracket \varphi \rrbracket_O(M, o) = \llbracket e \rrbracket_{\text{Core}}(M, \rho_o),$$

where φ is the source body and $\rho_o(\text{self}) = o$.

The proof is structural over the resolved surface syntax. Name resolution preserves declaration identity, normalization preserves binder scope, and collection lowering preserves Set support and Bag occurrences.

Theorem 10.2 (Core-to-Q preservation) *Assume $\text{Adequate}_\Sigma(M, G)$ and $\rho \simeq_\Sigma \eta$. Let $e \in \text{Expr}_{\text{Core}}[\tau]$ and $q \in \text{Expr}_Q[\text{Ty}_\Sigma(\tau)]$ satisfy $\Sigma \vdash e \rightsquigarrow q$. If every applicable semantic preservation obligation in Appendix A holds, then*

$$\llbracket e \rrbracket_{\text{Core}}(M, \rho) \simeq_\Sigma \llbracket q \rrbracket_Q(G, \eta).$$

The expression result is proved simultaneously with the corresponding result for $P_C \in \text{Plan}_{\text{Core}}$ and $P_Q \in \text{Plan}_Q$.

The proof is simultaneous induction on the expression and plan translation derivations. Constants, variables, and bottom are immediate. Attribute and navigation cases use the adequacy clauses. Iterator cases apply the induction hypothesis to every occurrence and then use the false/bottom/true priority of the three-valued fold. Set cases use support; Bag cases use occurrence counts. Numeric cases invoke the explicit domain obligations from Section 9.

10.3 Violation Projection

Lemma 10.3 (Violation-projection preservation) *Assume representation adequacy,*

$$\text{Adequate}_\Sigma(M, G),$$

and

$$\forall o \in \text{Inst}_M(K) : \llbracket e \rrbracket_{\text{Core}}(M, \rho_o) \simeq_\Sigma \llbracket q \rrbracket_Q(G, \eta_{\gamma(o)}).$$

Then

$$\{id_M(o) \mid o \in \text{Inst}_M(K), \llbracket e \rrbracket_{\text{Core}}(M, \rho_o) \neq \top\} = \text{Obs}_Q(G, K, q).$$

Proof. Adequacy pairs the source and graph extents and preserves stable IDs. Boolean correspondence preserves the predicate of being different from $\top$. The wrapper removes duplicate IDs, so both inclusions follow.

10.4 Presentation-Level Composition

For a source invariant inv , define the presentation-level name

$$\text{Compile}_{S,\Sigma}(inv) = \text{Success}(s, \Delta, \pi_{\text{gen}})$$

iff there are K, e, q satisfying

$$\begin{aligned} \text{Front}_S(inv) &= \text{Success}(K, e), \\ T_\Sigma(e) &= \text{Success}(q), \\ \text{Realize}_\Sigma(\text{Violations}(K, q)) &= \text{Success}(s, \Delta, \pi_{\text{gen}}). \end{aligned}$$

$\text{Compile}_{S,\Sigma}$ is only a name for this composition; it is not a new semantic layer.

10.5 Main Theorem

Theorem 10.4 (Conditional exact preservation) *Fix S, M, G, Σ and invariant inv . Let $(s, \Delta, \pi_{\text{gen}})$ be a compilation result. Assume:*

P1. Source admissibility: $\text{Conforms}(M, S) \wedge \text{WF}_S(M)$; M is finite, and inv belongs to the supported profile and is well typed under S .

- P2.** Representation adequacy: $\text{Adequate}_\Sigma(M, G)$.
- P3.** Compilation: $\text{Compile}_{S, \Sigma}(\text{inv}) = \text{Success}(s, \Delta, \pi_{\text{gen}})$. Fix K, e, q from the successful stages.
- P4.** Refinement: There is a p such that $\text{Witness}_\Sigma(p; G, K, q, s, \Delta, \pi_{\text{gen}})$. Every applicable semantic preservation obligation in Appendix A holds, and R1–R2 hold for p .
- P5.** Runtime: $\pi_{\text{public}} \in \text{Dom}_{\text{pub}}$, $\text{dom}(\pi_{\text{public}}) \cap \text{dom}(\pi_{\text{gen}}) = \emptyset$, and $\pi = \pi_{\text{public}} \uplus \pi_{\text{gen}}$. Every applicable runtime-dependent numeric-domain obligation holds, $\llbracket s \rrbracket_C(G; \pi, \Delta) = \text{Success}(I)$ with $I \subseteq \text{ID}$, and R3 holds for the same p, I .

Then

$$\boxed{\text{Viol}_O(M, \text{inv}) = \text{Viol}_C(G, s; \pi, \Delta).}$$

Proof. P1 and frontend preservation give the Core body e with the same source value at every context object. P2 supplies γ and the observer correspondence. P3 supplies the successful Core-to- Q translation; P4 and the numeric premises in P5 supply its semantic side conditions. Core-to- Q preservation and violation projection therefore give

$$\text{Viol}_O(M, \text{inv}) = \text{Obs}_Q(G, K, q).$$

The successful realization from P3, the witness and R1–R2 from P4, and the execution and R3 premises from P5 satisfy the generated-query refinement lemma of Section 8; hence

$$\text{Obs}_Q(G, K, q) = \text{Viol}_C(G, s; \pi, \Delta).$$

Transitivity yields the result. Equivalently, the first equality prevents a missing or spurious source ID before realization, while R1–R3 preserve exactly that set across the formal target, text, execution, and decoding boundaries.

10.6 Non-Circularity and Assurance Boundary

No admission checker compares Viol_O with Viol_C . P1 checks syntax, resolution, typing, and source well-formedness; P2 is an observational contract; P3 records successful compilation; P4 states semantic refinement obligations; and P5 states runtime premises. Finite differential and live experiments are evidence for production boundaries, not premises that define correctness.

The Lean development checks a formal normalization/Core-to- Q /target slice under its declared semantic algebra. It does not verify the production graph builder, Java compiler, emitted Cypher text, decoder, Neo4j implementation, or runtime numeric premises. Those boundaries retain their manual, conditional, or live-tested status in Appendix B.

11 Mechanization and Proof Assurance

This section records which parts of the argument are machine checked and which remain manual or implementation-level. The assurance vocabulary is deliberately explicit: a Lean build is not evidence that the concrete Java, Cypher, or Neo4j runtime has been verified.

11.1 Build and trusted base

The current mechanized development is built with Lean 4.33.1 using the project root module and ‘lake build Ocl2Gratra’. The trusted base is Lean’s kernel and the imported mathematical libraries. The current gate completed with exit code zero and 21 build jobs. The older empirical snapshot retained in Section 15 records its original Lean 4.32.2/13-job environment rather than relabelling that historical run.

The named declarations used by the main paper are:

```
p3_normalization, p3_core_to_q, p4_q_to_target,  
parse_serialize, p3_p4_composition,  
compiled_exact_violation_ids.
```

The more general `exact_violation_identifier_set` is conditional on explicit refinement witnesses. In particular, `parse_serialize` is a round trip theorem for the canonical formal artifact, not a proof about every text emitted by the production serializer.

Named result	Syntactic coverage	What is established
<code>p3_normalization</code> , <code>p3_core_to_q</code>	20 expression/7 plan constructors	Equality under the shared parameterized semantic algebra
<code>p4_q_to_target</code> , <code>p3_p4_composition</code>	The same 20/7 inventory	Equality with the independent formal target calculus
<code>parse_serialize</code>	Canonical Artifact wrapper	Formal round trip, not production Cypher text
<code>compiled_exact_violation_ids</code>	Eight-constructor Boolean surface slice	Exact IDs for that slice only

Constructor coverage is not operator-semantic coverage: fields such as numeric operations, observers, and collection primitives in `FullP3P4.Semantics` are operations of a common semantic algebra, not proof hypotheses that Java or Neo4j implements each operation correctly. Separate kernel families below establish selected primitive laws; the remaining production correspondences retain their stated status.

11.2 Coverage matrix

Family	Role in this report	Status
Boolean3Kleene	Three-valued tables and fold laws	kernel checked
CollectionPipeline	Set/Bag support and multiplicity	kernel checked
ExactRealField	finite real carrier conditions	kernel checked
IntegerRangeCertificate	interval soundness	kernel checked
OclEqualityTotal	typed equality cases	partial
OclTypeLattice	inheritance and type tests	partial
ProductionQSyntax	logical query syntax	partial
ProductionEndToEnd	theorem-shaped composition	conditional
ScalarCodec	value serialization	partial
DifferentialTheorem	local comparison abstraction	partial

In this legacy family ledger, four families are kernel-complete, five are partial, and the production end-to-end row is conditional on the explicit refinement premises in Sections 8–10. The separately named `FullP3P4` formal slice above is kernel checked under its own formal assumptions; it does not promote the production row to kernel-complete. The ledger contains 122 accepted theorem/lemma declarations, but that number is not a count of fully mechanized paper obligations.

11.3 Gaps and stale artefacts

The production realization and serialization refinement, concrete executor semantics, and release-specific Neo4j behavior remain outside the mechanized kernel. The current source tree contains no accepted `sorry`, `admit`, or project-specific axiom in the cited declarations; reported dependencies are limited to Lean’s trusted-base principles. The legacy proof-contract checker is not used as evidence because its module names and contract version are stale. These gaps are recorded as ‘PARTIAL’, ‘CONDITIONAL’, or ‘OUT-OF-SCOPE’, never hidden under a single ‘verified’ label.

12 Prototype Boundary

The prototype is a witness that the formal pipeline can be executed; it is not itself the subject of the theorem. This section maps abstract stages to implementation responsibilities while making refinement gaps visible.

12.1 Architecture

The data flow is

$$\begin{aligned} \text{surface OCL} &\rightarrow \text{frontend} \rightarrow \text{Core} \rightarrow \text{Q} \rightarrow \text{Cypher AST} \\ &\rightarrow \text{text and parameters} \rightarrow \text{decoder}. \end{aligned}$$

In parallel, a graph builder materialises a snapshot according to Σ . The executor is downstream of serialization and is therefore outside the formal translation function T_Σ .

12.2 Traceability

The frontend implements name resolution, typing, and normalization. The Core layer carries resolved declaration keys and typed bottoms. The Q layer implements observers, three-valued folds, and violation projection. The realizer and serializer enforce the read-only profile, generated-parameter namespace, and result contract: an IDS query yields zero or more stable-ID rows, while a VALUE query yields exactly one tagged-value row. The graph builder is responsible for stable IDs, direct types, slots, and keyed link occurrences. Each responsibility has a matching obligation in Appendix B.

12.3 Validation gates

There are three fail-closed gates. The frontend rejects unresolved or ill-typed terms. The translator rejects constructors without a profile rule or certificate. The execution/decoder boundary reports operational errors or malformed rows rather than returning an empty set. A successful path therefore carries an explicit status at every boundary.

12.4 Refinement gaps

The formal observers and the production data structures are related by a traceability argument, not by a machine-checked refinement theorem. In particular, concrete object allocation, Cypher AST construction, driver configuration, and Neo4j executor behavior are not assumed to be verified. Local differential tests and live deployment checks provide evidence for these boundaries and are reported as such in Sections 13–14.

13 Experimental Method

The evaluation tests semantic agreement and rejection behavior at defined boundaries. It does not estimate latency, throughput, or scalability.

13.1 Research questions

1. RQ1: On prototype-admitted inputs, do Core and Q agree on values, occurrence-counted collections, and violation IDs?
2. RQ2: Are unsupported or uncertified constructs rejected at the intended frontend/lowering boundary?
3. RQ3: Does the graph-to-Neo4j path return the same violation IDs as source OCL on the reported deployment?

13.2 Subjects and units

Subject	Definitions	Snapshots	Local	Live
Library	26	3	78	0
CarRental	25	1	19	19
DeepHierarchy	12	3	36	0
Medical	36	1	36	36
Feature matrix	30	1	30	30
Boundary corpus	15	–	rejected	–

The unit of analysis is a definition–snapshot evaluation. A local comparison contains source, Core, and Q observations; a live comparison adds the generated Cypher execution. Rejections are counted separately and never enter the agreement denominator.

Prototype admission is broader than the theorem profile in Section 3. The feature matrix also exercises implementation extensions, including **reject**, Set operations, **allInstances**, and type tests/casts. Those rows test the implementation only. Aggregate RQ counts therefore must not be read as enlarging the theorem; only profile-conforming rows are direct evidence for its published boundary.

13.3 Oracles and coverage

The source oracle evaluates the frontend-produced Core expression directly on the snapshot. It shares frontend/lowering and the schema, snapshot, and value representations, but bypasses Q , graph construction, realization, serialization, Neo4j, and decoding. Core and Q results are compared after separate stable-ID projection paths; this is not implementation independence. Coverage is tagged by semantic risk: reverse navigation; empty versus bottom; Set/Bag multiplicity; nested iterators; inheritance extent; and unsafe numerics. The numeric risk is represented by intentional rejection, not by a successful comparison without a certificate. Shared fixtures and carriers are a construct-validity threat and are disclosed rather than treated as independent implementations.

13.4 Live protocol

The live gates materialise the CarRental, Medical, and feature-matrix snapshots in the configured database, run each generated query, decode its IDS rows or tagged VALUE row according to the result contract, and clean the disposable data. The verified endpoint ran Neo4j 2026.06.0 Enterprise and database ‘demo’. The 85 live comparisons comprise 19 CarRental, 36 Medical, and 30 feature-matrix comparisons. Wall-clock Maven time is not reported as query latency.

14 Results

All counts in this section are taken from the evidence manifest snapshot dated 2026-09-13. They count semantic cases, not source files or generated rows.

14.1 Headline outcomes

Measure	Count
Invariant definitions admitted by the prototype	129
Successful realization cases	110
Invariant-snapshot outcomes	205
Core-Q differential comparisons	199
Explicit rejections	23
Fresh live Neo4j comparisons	85
Local target-only / source-only ID sets	0 / 0
Live target-only / source-only ID sets	0 / 0

The 199 local comparisons and 85 live comparisons agreed on violation IDs. The full Maven regression reported 426 tests with zero failures and zero errors; five Maven-summary tests were skipped. These results support the reported prototype behavior. They do not establish theorem-profile coverage for rows identified as implementation extensions.

14.2 Risk-oriented outcomes

Reverse navigation, empty-versus-bottom behavior, Set/Bag occurrence semantics, nested iterators, and inheritance extents all have local evidence; the feature matrix also exercises these risks and implementation-only extensions live. Unsafe numerics have no positive comparison: the ‘GradedBonus’ case is rejected because its certificate premise is unavailable. The boundary corpus contributes 15 additional intentional rejections, while six CarRental frontend cases and one unsupported feature case account for the remaining rejection counts.

14.3 Threats to validity

Internal validity: source and query evaluators share fixtures and some typed carriers, so correlated defects cannot be excluded.

Construct validity: stable-ID equality measures the public observation, not internal query plans or all values.

External validity: subjects are artifact case studies and synthetic hierarchy fixtures; results do not generalise to arbitrary UML/OCL or graph encodings.

Conclusion validity: zero mismatches in a finite sample is not a probability statement and does not replace the conditional theorem.

Runtime validity: live agreement is for the recorded Neo4j 2026.06.0 Enterprise deployment and configured database ‘demo’; it is neither evidence for other versions/configurations nor universal Neo4j conformance.

No latency, throughput, graph-size, or scalability conclusion is drawn. Older historical counts are quarantined because they used different units or source snapshots.

15 Reproducibility and Traceability

This section defines the artifact identity and the checks needed to reproduce the report’s claims. Exact commands are paired with expected exit status; no secret or endpoint credential is part of the artifact.

15.1 Evidence snapshot and environment

The empirical evidence snapshot is dated 2026-09-13. Its machine-readable records are stored in the evidence directory

```
ocl2cypher/target/experiments/  
20260913-010523-6e3658498d  
064ec8ac6fcec833a59fb0.
```

The empirical snapshot recorded Oracle JDK 23.0.2, Apache Maven 3.9.9, Lean 4.32.2, and generated read-only Cypher 5. The live endpoint ran Neo4j 2026.06.0 Enterprise against configured database ‘demo’. Its manifest records revision 780dc553 (full hash in the manifest) plus per-file hashes in `inputs-before.json`; the commit alone does not identify uncommitted inputs. The separate legacy `verification/report/proof-report.json` records source revision 918dafdb (full hash in that record) and a NOT_RUN verification gate. It is not evidence that the present Lean modules or the 85-case live gate ran at that revision. An earlier clean runtime capture at 737c552c addresses different contract cases and is not added to the 199/23/85 totals. The current proof build was performed on the working tree at HEAD e7e40aaf, which contains uncommitted Lean changes; its provenance is therefore not a reproducible commit by itself.

The public artifact repository is

<https://github.com/bao2811/TGG0CL2Cypher>.

No DOI has been assigned to this report as of version 0.3. A DOI, release tag, or archived deposit must replace this explicit ‘not assigned’ status before a venue requires a permanent bibliographic identifier. The report edits after the frozen empirical run do not relabel that run as newly generated evidence. The repository URL is a discovery pointer, not a frozen release; a citable release must bundle the source snapshot, the manifest, and the generated queries before the empirical claims are independently reproducible from a single identifier.

15.2 Reproduction gates

Run the following commands from the repository root unless another working directory is stated. Here `<run>` denotes a new writable evidence directory.

Regression (exit 0). `mvn -pl ocl2cypher test -Docl2cypher.
testReportsDirectory=<run>/regression`

must report 426 tests, zero failures/errors, and five Maven-summary skips.

CarRental live (exit 0).

`mvn -pl ocl2cypher -Dtest=CarRentalNeo4jReplayTest test -
Docl2cypher.testReportsDirectory=<run>/neo4j`

must report 19/19 comparisons.

Medical live (exit 0).

`mvn -pl ocl2cypher -Dtest=MedicalNeo4jReplayTest test -
Docl2cypher.testReportsDirectory=<run>/neo4j-medical`

must report 36/36 comparisons.

Feature matrix live (exit 0).

`mvn -pl ocl2cypher -Dtest=Neo4jFeatureMatrixTest test -
Docl2cypher.testReportsDirectory=<run>/neo4j-feature-matrix`

must report 30/30 comparisons.

Lean (exit 0). `cd verification/lean
lake build Ocl2Gratra`

The frozen snapshot reports 13 build jobs. For the current source revision, the same gate has also been rerun with Lean 4.33.1 and reports 21 build jobs; this newer proof build does not retroactively change the empirical snapshot.

All three live commands require a disposable database and explicit cleanup. The legacy proof-contract checker is expected to fail because it references stale modules/contract metadata and is out of scope for the Ocl2Gratra gate.

15.3 Artifacts and regeneration

The raw inputs and machine-readable outcomes are stored in the evidence directory above. `inputs-before.json`, `manifest.json`, `outcome.json`, the three live-report directories, and the regression reports record the gates. Tables in this report must be regenerated from those records rather than edited independently. A release package should include the evidence manifest, generated query listings, rejection diagnostics, and exact environment-file schema with secrets removed. Any source change that affects a positive claim requires a fresh evidence snapshot and rerun of the corresponding gate.

15.4 Interpretation of failures

An exit code different from zero, a changed count, or a missing result-contract observation blocks publication of the corresponding positive claim. A parse, execution, parameter, or decoder failure is not an empty violation set. Re-running a live gate must use a disposable database; it must not modify a production instance. Reproducibility demonstrates the finite evidence snapshot and does not enlarge the mathematical theorem or its admitted language.

A Core-to-Q Translation Rules

This appendix expands the Core-to- Q families from the conference paper. It does not add constructors to the theorem profile. For every row, $\Sigma \vdash t \rightsquigarrow t'$ denotes $T_\Sigma(t) = \text{Success}(t')$; semantic side conditions that are not discharged by compiler success remain premises of preservation.

Core form	Q form	Admission and preservation obligation
$c_\tau, \perp_\tau, x$	Same typed constant, bottom, or variable	Declared type and corresponding environment; numeric constants require the applicable domain certificate.
$\text{Let}(x, e_1, e_2)$	$\text{Let}(x, T_\Sigma(e_1), T_\Sigma(e_2))$	Fresh well-typed binder; a bottom value is bound rather than treated as translation failure.
$\text{If}(c, t, f)$	$\text{If}(T_\Sigma(c), T_\Sigma(t), T_\Sigma(f))$	Boolean condition, compatible branch type, lazy branches, and typed-bottom condition behavior.

Core form	Q form	Admission and preservation obligation
$\text{Attr}(e, a)$	$\text{Attr}(T_\Sigma(e), a)$	Resolved key and declared result type; slot adequacy and codec exactness.
$\text{NavOne}(e, r)$	$\text{NavOne}(T_\Sigma(e), r)$	Resolved direction/type; exactly one target is defined, while zero or multiple targets yield typed bottom.
$\text{NavMany}_\kappa(e, r)$	$\text{NavMany}_\kappa(T_\Sigma(e), r)$	Resolved direction/type/kind; no occurrence gives a defined empty collection, Set uses support, and Bag preserves occurrences.
$\text{Un}_{o_1}(e)$	$\text{Un}_{o_1}(T_\Sigma(e))$	Boolean negation, certified numeric unary operations, or ‘isEmpty’, ‘notEmpty’, ‘size’, and ‘sum’; preserve typed-bottom behavior.
$\text{Bin}_{o_2}(e_1, e_2)$	$\text{Bin}_{o_2}(T_\Sigma(e_1), T_\Sigma(e_2))$	Boolean, typed equality/comparison, or certified numeric binary operation; runtime-dependent numeric-domain obligations remain P5 premises.
$\text{Lit}_\kappa(\bar{e})$	$\text{Lit}_\kappa(T_\Sigma(\bar{e}))$	Finite flat common element type; bottom elements remain elements of a defined collection.
$\text{From}(e)$	$\text{FromCollection}(T_\Sigma(e))$	The expression has a finite flat Set/Bag type; whole-bottom remains distinct from empty.
Scan_K	ScanClass_K	Resolved context key; inheritance-aware extent adequacy and one occurrence per object.
$\text{Select}(P, x, b)$	$\text{Filter}(T_\Sigma(P), x, T_\Sigma(b))$	Fresh Boolean body; retained occurrences and collection kind are preserved.
$\text{Collect}(P, x, e)$	$\text{Collect}(T_\Sigma(P), x, T_\Sigma(e))$	Fresh binder and non-collection body; result is a flat Bag preserving source occurrences.
$\text{Distinct}(P)$	$\text{Distinct}(T_\Sigma(P))$	Equality is defined for the element carrier; duplicate occurrences are removed by support.
$\text{LetP}(x, e, P)$	$\text{PlanLet}(x, T_\Sigma(e), T_\Sigma(P))$	Fresh well-typed binder; bind once and preserve plan occurrences.
$\text{Materialize}(P)$	$\text{Materialize}(T_\Sigma(P))$	Preserve collection kind, occurrences, and whole-bottom.
$\text{ForAll}(P, x, b)$	$\text{ForAll3}(T_\Sigma(P), x, T_\Sigma(b))$	Finite source and fresh Boolean body; false dominates bottom, empty is true, and a whole-bottom collection yields bottom.
$\text{Exists}(P, x, b)$	$\text{Exists3}(T_\Sigma(P), x, T_\Sigma(b))$	Finite source and fresh Boolean body; true dominates bottom, empty is false, and a whole-bottom collection yields bottom.

Core form	Q form	Admission and preservation obligation
$\text{Violations}(K, q)$	Same top-level wrapper	Scan $\text{Extent}_G^\Sigma(K)$, retain results different from T , and project DISTINCT stable IDs.

A.1 Explicit Surface Rejections

Surface family	Status	Reason
any	Rejected before Core	No deterministic witness-choice policy in the admitted semantics.
reject and unsupported iterators	Rejected before Core	No constructor in the theorem profile.
allInstances , type tests, and casts	Rejected before Core	The profile exposes inheritance only through context extents and navigation.
Membership/count, Set union/intersection, asSet/asBag	Rejected before Core	No admitted Core/ Q constructor and preservation rule.
Ordered or nested collections	Rejected before Core	Order and nested-carrier observations are outside the theorem.
Tuple values or recursion	Rejected before Core	No closed typed translation rule.
Surface null/invalid	Rejected before Core	Typed bottom does not claim the full OMG distinction.
Uncertified numeric operation	Rejected or retained as an explicit premise	Required range, overflow, finite-real, or rounding obligation is absent.

Failure of a compiler-checked condition produces a diagnostic and no Q term. A semantic or runtime condition not checked by the compiler is stated as a premise; it is not inferred from successful source/target agreement.

B Representative Proof-Obligation Ledger

The ledger distinguishes assurance levels rather than using one overloaded word such as “verified.”

ID	Obligation	Method	Status
O-01	Production frontend resolution, typing, and normalization	manual + regression	PROVED-MANUAL
O-02	Formal normalization slice	Lean kernel	LEAN-CHECKED

O-03	Abstract construction establishes observational adequacy	invariant proof	PROVED-MANUAL
O-04	Production graph builder satisfies adequacy	finite adequacy checks	TEST-SUPPORTED
O-05	Three-valued Boolean and collection laws	Lean kernel	LEAN-CHECKED
O-06	Formal numeric-certificate soundness	Lean kernel	LEAN-CHECKED
O-07	Formal Core-to-Q preservation slice	Lean kernel	LEAN-CHECKED
O-08	Production Core-to-Q compiler refinement	mutual hand argument + tests	PARTIAL
O-09	Formal Q-to-target preservation slice	Lean kernel	LEAN-CHECKED
O-10	Production target/Cypher realization, R1–R2	hand refinement + tests	PARTIAL
O-11	Formal artifact serialization round trip	Lean kernel	LEAN-CHECKED
O-12	Production Cypher serialization	parser acceptance + structural checks	PARTIAL
O-13	General violation projection	set argument	PROVED-MANUAL
O-14	Exact IDs for the mechanized surface slice	Lean kernel	LEAN-CHECKED
O-15	Concrete Neo4j execution/decoder, R3	live comparisons	LIVE-TESTED
O-16	Universal engine conformance	not established	OUT-OF-SCOPE

The paper evidence snapshot records 37 closed hand-proof rows. This appendix summarizes representative obligations in the dependency chain used in Section 10; the remaining rows are homomorphic operator instances or explicit rejection obligations. It is therefore a report-level summary, not a reproduction of all 37 row-level derivations. ‘OPEN’ is reserved for an obligation with no accepted evidence; no such row is silently promoted to a positive theorem premise.

C Generated Cypher Examples

This appendix records translator-produced artifacts rather than handwritten pseudo-Cypher. Each successful example was elaborated, normalized, translated to Q, realized as a Cypher AST, serialized, and accepted by the Cypher 5 parser gate. Source and Q-over-*G* observations were then compared on the same snapshot. The Manager and Bag examples also occur in the live CarRental/feature-matrix coverage reported in Section 14; the compact Library attribute example is a local source/Core/Q and parser-acceptance witness, not a claimed live case.

The complete generated texts are distributed beside this report in the `evidence/` directory. This is necessary for `ManagerSalaryDominates`: its scope-bound, typed-bottom-preserving text is 7,731 bytes and 14 lines. Reprinting the complete expression inside the PDF would still obscure rather than improve inspection. The ledger gives each external listing’s filename, byte count, and physical line count.

Artifact	Bytes	Lines
Manager	7,731	14
Bag	4,573	12
Attribute	2,276	6

C.1 ManagerSalaryDominates: Reverse Navigation and Three-Valued Fold

Source schema and invariant. `Employee::salary` has type Integer. A Branch observes `employee:Set(Employee)` and `manager:Employee[0..1]`; both logical roles reverse physical Employee-to-Branch link occurrences.

```
context Branch inv ManagerSalaryDominates:
  self.employee->forAll(e |
    e <> self.manager implies
    self.manager.salary > e.salary)
```

Snapshot and graph fragment. The snapshot contains `branch_bad`, Alice with salary 50, and Bob with salary 70. Its employee Set is {Alice, Bob} and its manager is Alice. The construction produces these three relevant physical occurrences:

```
Alice -[employment, sourceRole=employee]-> branch_bad
Bob   -[employment, sourceRole=employee]-> branch_bad
Alice -[management, sourceRole=manager] -> branch_bad
```

All three are physically incoming at the Branch receiver. The generated text therefore contains incoming `LinkAssociateWith` patterns for both roles, exact-cardinality guards for `manager`, typed slot decoding, a Boolean₃ fold, and a final distinct stable-ID projection.

Generated parameters. All parameters are generated; there are no public parameters:

Name	Logical type	Canonical value
<code>__oclContextClassKey</code>	Physical:ClassKey	Branch
<code>__oclModelKey</code>	Physical:ModelKey	carrental
<code>__oclRole_2</code>	Physical:Role	employee
<code>__oclRole_9,</code>	Physical:Role	manager
<code>__oclRole_16</code>		
<code>__oclAttr_21,</code>	Physical:AttributeKey	Employee::salary
<code>__oclAttr_27</code>		

Exact generated-text excerpt and complete artifact. The following first two lines are read directly from the archived translator output, not reconstructed by hand. The complete query is the Manager artifact identified by filename and size in the ledger above.

```
CYPHER 5
MATCH ('self': 'Object' { 'modelKey': $__oclModelKey }) -[: 'ObjectInstanceOf' ] -> (
  directClass_1: 'UmlClass' { 'modelKey': $__oclModelKey }) -[: 'Extends' *0..11] -> (
    c_0: 'UmlClass' { 'modelKey': $__oclModelKey, 'classKey': $__oclContextClassKey
  })
```

Observations. The source and Q interpreters returned, respectively,

source IDs = {branch_bad}, Q-over-*G* IDs = {branch_bad}.

The serialized text passed the Cypher 5 parser gate, and its violation-ID result matched the source oracle in the live CarRental replay.

C.2 Bag Multiplicity

Source, snapshot, and graph. The role `Company::employees` is a non-unique Bag-valued association end. The invariant is

```
context Company inv ThreeOccurrences:
  self.employees->size() > 2
```

The snapshot contains `acme`, `empty`, `e1`, and `e2`; the graph has two separately keyed `acme -> e1` occurrences and one `acme -> e2` occurrence. Thus the Bag size for `acme` is three rather than two.

Parameters and observations. The generated parameter map contains

```
__oclContextClassKey ↦ Company,
__oclRole_2 ↦ employees.
```

Both the source and Q-over-*G* observations are {empty}. The following first two lines are an exact excerpt; the complete parser-accepted Bag artifact is the file identified by the ledger. Keeping the 4,573-byte query external avoids an unreadably small full-page listing.

```

CYPHER 5
MATCH ('self':Object' {'modelKey': 'bag-report'})-[:ObjectInstanceOf]->('
    directClass_1':UmlClass' {'modelKey': 'bag-report'})-[:Extends'*0..2]->('c_0
    ':UmlClass' {'modelKey': 'bag-report', 'classKey': $_oclContextClassKey})

```

C.3 Rejected Numeric Case

The evidence corpus contains the source invariant

```

context Person inv GradedBonus:
    let a : Integer = self.age in
    if a > 20 then a + 10 else 5 endif >= 18

```

For the main Library snapshot its source violation IDs are {bob, carol}. Realization returns diagnostic category R_NUMERIC_CAPABILITY because the required integer range certificate is unavailable. Hence there is deliberately no generated Cypher, no parameter map, and no target violation-ID observation for this row. Treating that rejection as an empty Set would contradict the partial target observation used by the main theorem.

C.4 Compact Compiler-Generated Attribute Read

This compact example was generated from the production pipeline, not written as schematic Cypher. The source invariant is

```

context Book inv NonEmptyTitle: self.title <> "

```

The Library SN1 snapshot contains book1 with a non-empty title and bookBlank with the empty string. Source and Q-over-G observations both return {bookBlank}. The generated parameter map contains __oclModelKey=library, __oclContextClassKey=Book, and __oclAttr_2=Book::title. The complete serializer output uses the canonical object/type scan, the reified ObjectHasAttribute -> AttributeValue slot path, codec/state guards, tagged String/Boolean3 carriers, and final use_id projection. To keep the PDF readable, the generated CYPHER 5 header and four body lines are omitted from the display below. The exact final projection is read directly from the artifact; the full 2,276-byte output remains in the evidence directory.

```

RETURN DISTINCT 'self'.use_id AS 'violationId'

```

The complete external text was accepted by the Cypher 5 parser gate and is auditable in the same way as the three larger artifacts.

D Experimental Inventory and Catalogue Schema

This appendix is an aggregate experimental inventory plus the schema required for a row-level catalogue. It does not claim to reproduce every one

of the 205 definition–snapshot outcomes in the PDF. The machine records named below are the authoritative row-level evidence for this report version.

A normalized catalogue row has the fields: subject, definition identifier, snapshot identifier, feature tags, expected source IDs, local target IDs, live target IDs, status, and diagnostic code (when rejected). The evidence directory is

```
ocl2cypher/target/experiments/
20260913-010523-6e3658498d
064ec8ac6fcec833a59fb0
```

Its `inputs-before.json`, `manifest.json`, `outcome.json`, and archived Surefire XML/text reports identify the inputs, toolchain, commands, individual tests, and gate outcomes.

Status	Meaning	Count	Expected relation
Local	source/Core/Q comparison	199	target-only = source-only = 0
Live	generated Cypher deployment	85	target-only = source-only = 0
Rejected	admission/lowering boundary	23	diagnostic required

Subject	Local	Live	Inventory note
Library	78	0	26 definitions over three snapshots; 25 realizations accepted and one numeric case rejected
CarRental	19	19	25 definitions; six frontend rejections excluded from the agreement denominator
DeepHierarchy	36	0	12 definitions over three synthetic snapshots
Medical	36	36	36 definitions; all local and live comparisons passed
Feature matrix	30	30	30 focused definitions spanning theorem-profile risks and prototype-only extensions; one separate unsupported operation rejected
Boundary corpus	–	–	15 rejection-only cases with diagnostics
Total compared	199	85	zero target-only and source-only IDs

Feature tags identify reverse navigation, empty/bottom, Set/Bag, nested-iterator, inheritance, or numeric risk. Prototype admission is broader than the theorem profile: feature-matrix rows using `reject`, Set operations, `allInstances`, or type tests/casts are implementation-only evidence and

do not enlarge the theorem. The 129 prototype-admitted definitions are $26 + 25 + 12 + 36 + 30$; the 15-item boundary corpus is rejection-only. The 205 definition-snapshot outcomes are $78 + 25 + 36 + 36 + 30$. Successful realization counts have a different unit and total 110. Consequently none of these totals may be substituted for another in derived tables.

An archival release should export the normalized row schema above as a single CSV/JSON file. Until that packaging step is performed, this appendix is titled an inventory and catalogue schema, not a complete test catalogue.

References

- [1] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Alastair Green, Jan Hidders, Bei Li, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Stefan Plantikow, Ognjen Savković, Michael Schmidt, Juan Sequeda, Sławek Staworko, Dominik Tomaszuk, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Dušan Živković. PG-Schema: Schemas for property graphs. *Proc. ACM Manag. Data*, 1(2):1–25, June 2023. Art. no. 198.
- [2] Amine Benelallam, Abel Gómez, Gerson Sunyé, Massimo Tisi, and David Launay. Neo4EMF: A scalable persistence layer for EMF models. In *Proc. 10th Eur. Conf. Model. Foundations Appl.*, volume 8569 of *Lect. Notes Comput. Sci.*, pages 230–241, 2014.
- [3] Gwendal Daniel. Efficient validation of large models using the Mogwai tool. In *Proc. 18th Int. Workshop on OCL and Textual Model. (OCL)*, volume 2245 of *CEUR Workshop Proc.*, pages 187–193, 2018.
- [4] Gwendal Daniel, Frédéric Jouault, Gerson Sunyé, and Jordi Cabot. Gremlin-ATL: A scalable model transformation framework. In *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, pages 462–472. IEEE, 2017.
- [5] Gwendal Daniel, Gerson Sunyé, Amine Benelallam, Massimo Tisi, Yoann Vernageau, Abel Gómez, and Jordi Cabot. NeoEMF: A multi-database model persistence framework for very large models. *Sci. Comput. Program.*, 149:9–14, 2017.
- [6] Gwendal Daniel, Gerson Sunyé, and Jordi Cabot. Mogwai: A framework to handle complex queries on large models. In *Proc. 10th Int. Conf. Research Challenges Inf. Sci. (RCIS)*, pages 1–12. IEEE, 2016.
- [7] Gwendal Daniel, Gerson Sunyé, and Jordi Cabot. UMLtoGraphDB: Mapping conceptual schemas to graph databases. In *Proc. 35th Int. Conf. Conceptual Model. (ER)*, volume 9974 of *Lect. Notes Comput. Sci.*, pages 430–444. Springer, 2016.

- [8] Marina Egea and Carolina Dania. SQL-PL4OCL: An automatic code generator from OCL to SQL procedural language. *Softw. Syst. Model.*, 18(1):769–791, 2019.
- [9] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. Cypher: An evolving query language for property graphs. In *Proc. 2018 Int. Conf. Manag. Data*, pages 1433–1445. ACM, 2018.
- [10] Martin Gogolla, Fabian Büttner, and Mark Richters. USE: A UML-based specification environment for validating UML and OCL. *Sci. Comput. Program.*, 69(1–3):27–34, 2007.
- [11] Yang He, Ruijie Fang, Işıl Dillig, and Yuepeng Wang. Graphiti: Bridging graph and relational database queries. *Proc. ACM Program. Lang.*, 9(PLDI):1–25, June 2025. Article 216.
- [12] Ana León, Maribel Yasmina Santos, Alberto García, Juan Carlos Casamayor, and Oscar Pastor. Model-to-model transformation: From UML class diagrams to labeled property graphs. *Bus. Inf. Syst. Eng.*, 66(1):85–110, 2024.
- [13] Hoang Nguyen Phuoc Bao and Manuel Clavel. OCL2PSQL: An OCL-to-SQL code-generator for model-driven engineering. In *Proc. Int. Conf. Future Data and Security Eng. (FDSE)*, volume 11814 of *Lect. Notes Comput. Sci.*, pages 185–203, 2019.
- [14] Hoang Nguyen Phuoc Bao and Manuel Clavel. Proving correctness for SQL implementations of OCL constraints, 2024. arXiv:2403.18599 [cs.DB].
- [15] Object Management Group. Object constraint language, version 2.4. Technical Report formal/2014-02-03, Object Management Group, February 2014.
- [16] Hendrik Radke, Thorsten Arendt, Jan Steffen Becker, Annegret Habel, and Gabriele Taentzer. Translating essential OCL invariants to nested graph constraints for generating instances of meta-models. *Sci. Comput. Program.*, 152:38–62, 2018.